Assignment 1: Build the Model from Scratch

March 19, 2026

SHAO Junjie | 314551819

GitHub link: https://github.com/2ec3/DM2026-Assignment-1.git

1. Linear Regression (20 points):

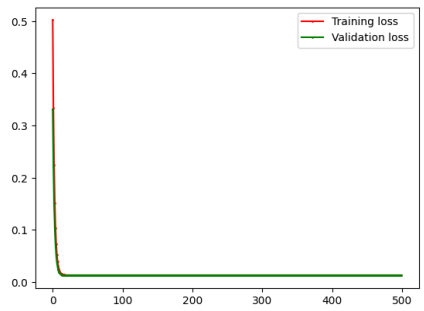
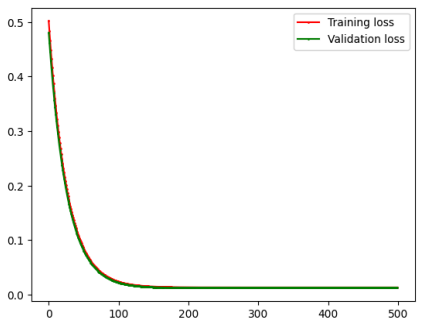
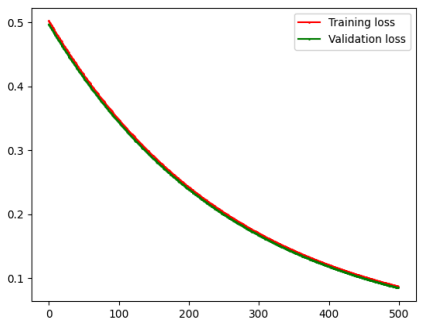
Please refer to **Linear_Regression.ipynb** and file(**metrics.py**) in the model folder. **Do not change random_seed**, as it will affect the evaluation results.

Apply the following questions 1(a) to 1(c) to **all datasets** (linear_data_A, linear_data_B, linear_data_C, and linear_data_D) in the data/linear_data folder.

a. **Change the learning rate to the following values, and plot the loss curve for each learning rate:**

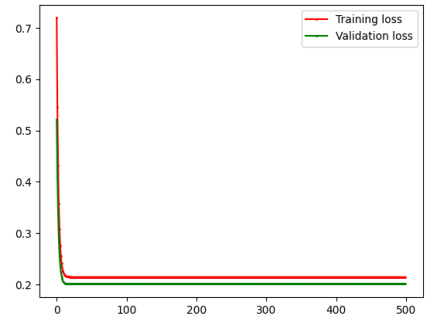
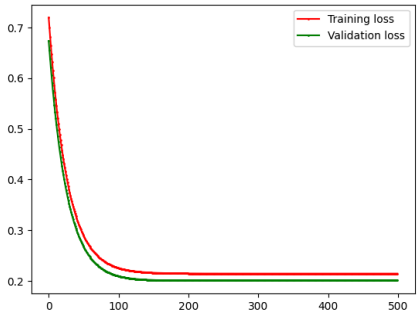
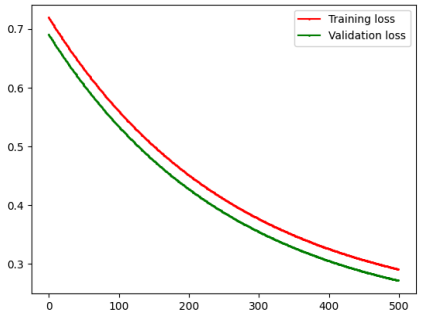
$$\text{learning rate} \in \{0.1, 0.01, 0.001\}$$

linear_data_A

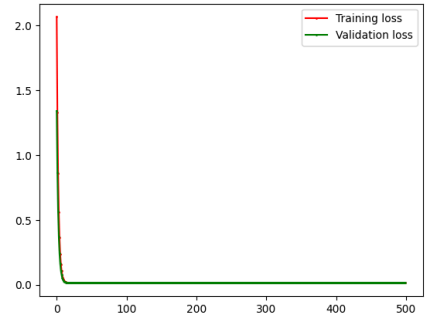
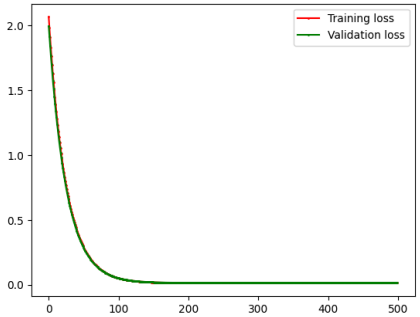
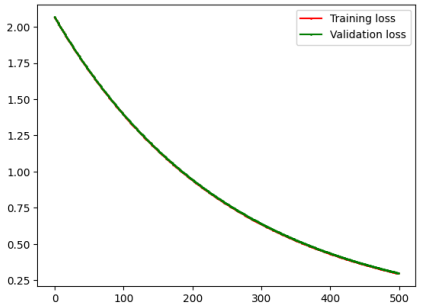
0.1	0.01	0.001
		

linear_data_B

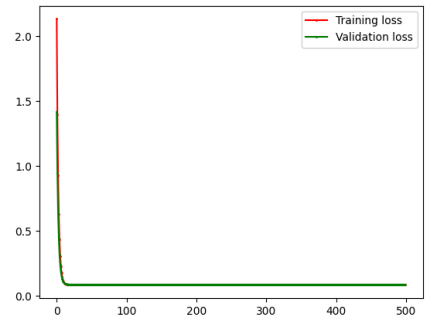
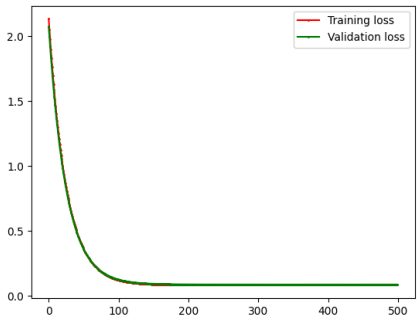
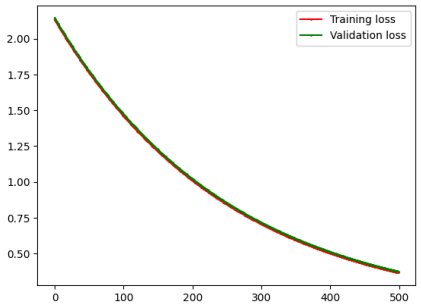
0.1	0.01	0.001

linear_data_C

0.1	0.01	0.001
		

linear_data_D

0.1	0.01	0.001
		

b. Following 1(a), show the evaluations by using the "evaluate_linear_regression" function:

linear_data_A

0.1	0.01	0.001
MSE : 0.0134 MAE : 0.1010 RMSE : 0.1156 R-squared : 0.5690	MSE : 0.0134 MAE : 0.1010 RMSE : 0.1156 R-squared : 0.5690	MSE : 0.0832 MAE : 0.2369 RMSE : 0.2884 R-squared : -1.6836

linear_data_B

0.1	0.01	0.001
MSE : 0.2137 MAE : 0.4038 RMSE : 0.4623 R-squared : 0.0448	MSE : 0.2137 MAE : 0.4038 RMSE : 0.4623 R-squared : 0.0448	MSE : 0.2687 MAE : 0.4404 RMSE : 0.5184 R-squared : -0.2011

linear_data_C

0.1	0.01	0.001
MSE : 0.0132 MAE : 0.0987 RMSE : 0.1148 R-squared : 0.9954	MSE : 0.0132 MAE : 0.0987 RMSE : 0.1148 R-squared : 0.9954	MSE : 0.2852 MAE : 0.4273 RMSE : 0.5341 R-squared : 0.8994

linear_data_D

0.1	0.01	0.001
MSE : 0.0824 MAE : 0.2467 RMSE : 0.2870 R-squared : 0.9716	MSE : 0.0824 MAE : 0.2467 RMSE : 0.2870 R-squared : 0.9716	MSE : 0.3540 MAE : 0.4778 RMSE : 0.5950 R-squared : 0.8780

c. Describe your observations in 1(a) and 1(b).

We can see in 1(a) that the loss is the highest for linear_data_B (when we look at the y-axes, the smallest value is close to 0.2, while for the other examples it is close to 0). And indeed, the difference is confirmed by the 1(b) stats (MSE is used as the loss in the figures).

Plus, we notice that the higher the learning rate, the smaller the loss. This makes sense since the model learns faster with a higher learning rate; thus, it fits the training data faster, which leads to a smaller loss.

2. Logistic Regression (20 points):

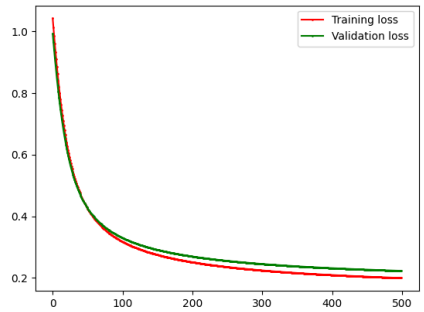
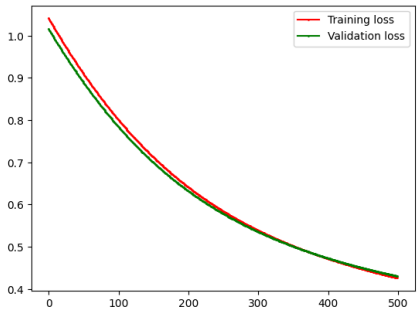
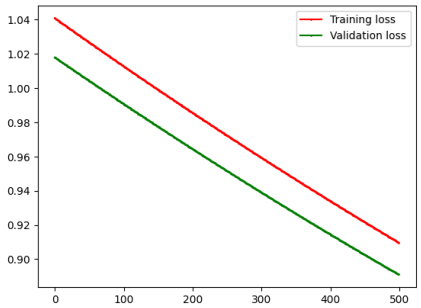
Please refer to **Logistic_Regression.ipynb** and file(**metrics.py**) in the model folder. **Do not change random_seed**, as it will affect the evaluation results.

Apply the following questions 2(a) to 2(f) to **all datasets** (logistic_data_A, logistic_data_B, logistic_data_C, and logistic_data_D) in the data/logistic_data folder.

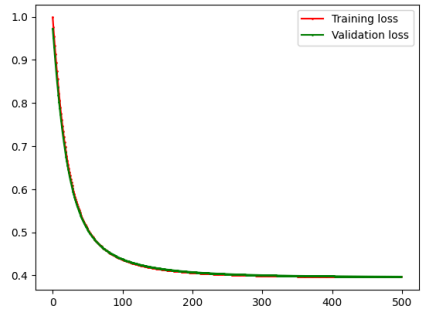
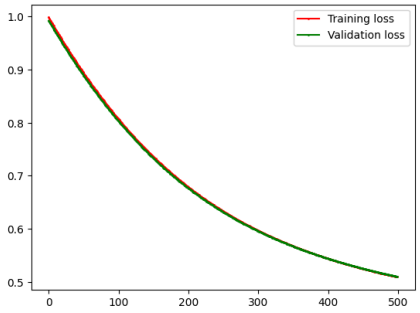
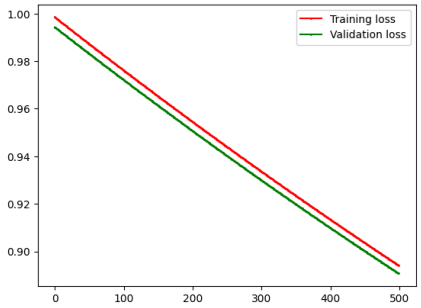
a. **Change the learning rate to the following values, and plot the loss curve for each learning rate:**

$$\text{learning rate} \in \{0.1, 0.01, 0.001\}$$

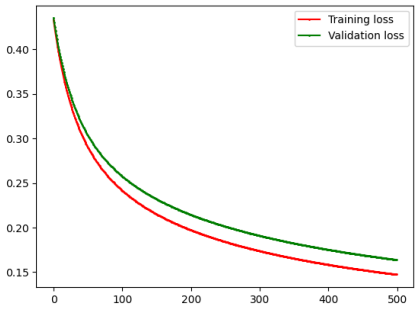
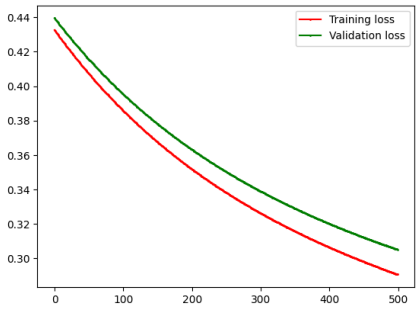
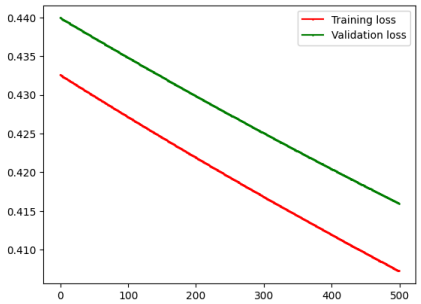
logistic_data_A

0.1	0.01	0.001
		

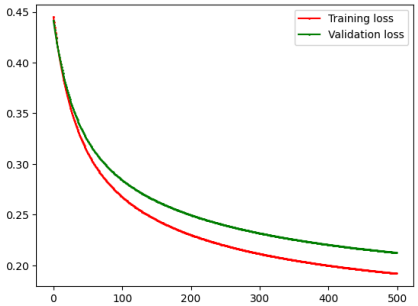
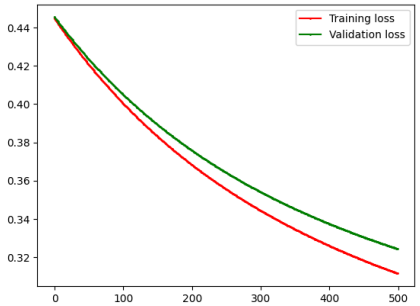
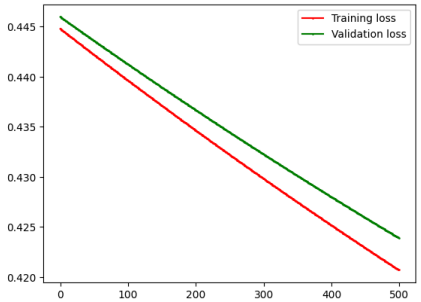
logistic_data_B

0.1	0.01	0.001
		

logistic_data_C

0.1	0.01	0.001
		

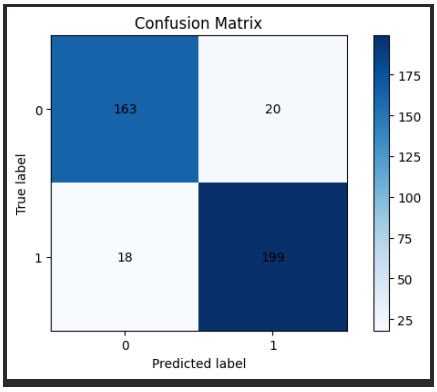
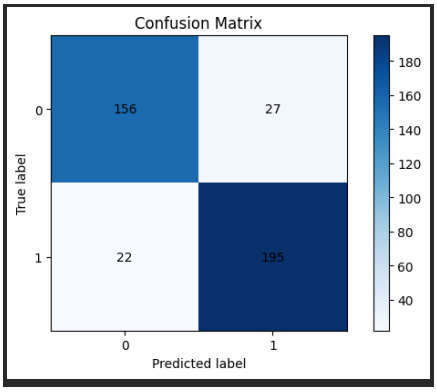
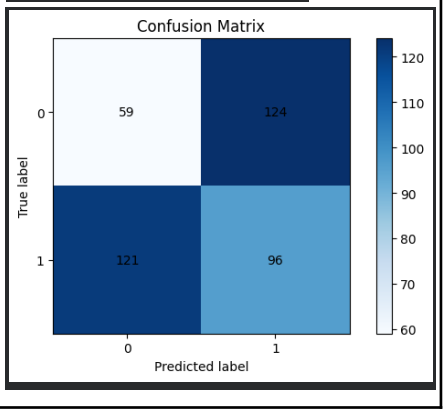
logistic_data_D

0.1	0.01	0.001
		

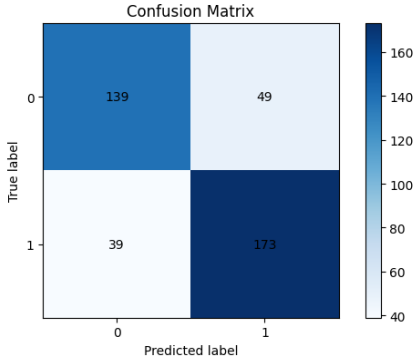
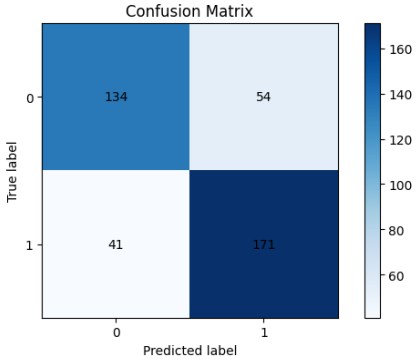
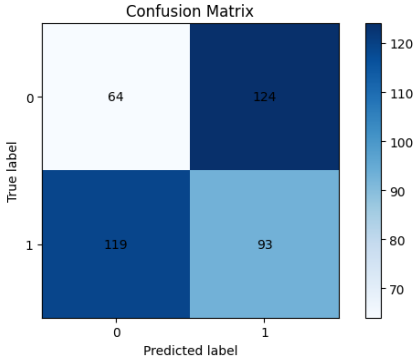
b. Following 2(a), show the evaluations by using the "evaluate_binary_classifier" function.

logistic_data_A

0.1	0.01	0.001
Accuracy : 0.9050 Precision : 0.9087 Recall : 0.9171	Accuracy : 0.8775 Precision : 0.8784 Recall : 0.8986	Accuracy : 0.3875 Precision : 0.4364 Recall : 0.4424

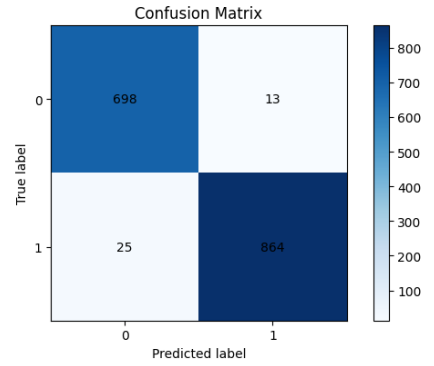
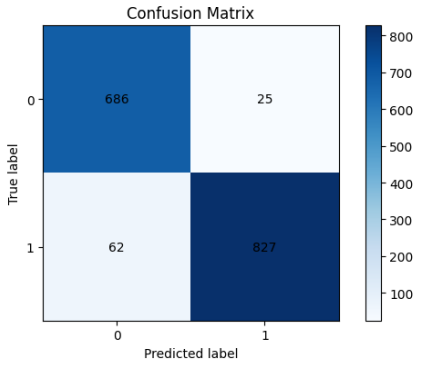
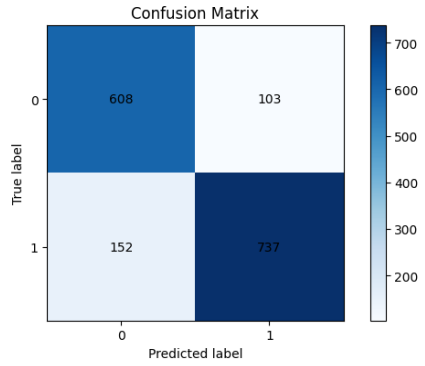
F1-score : 0.9128 	F1-score : 0.8884 	F1-score : 0.4394 

logistic_data_B

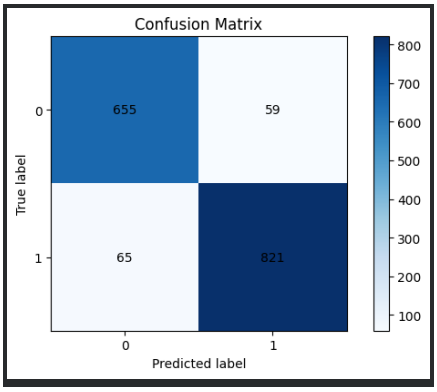
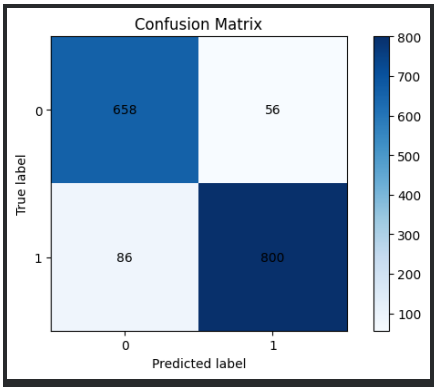
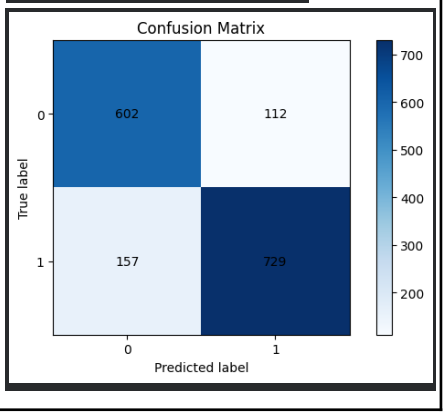
0.1	0.01	0.001
Accuracy : 0.7800 Precision : 0.7793 Recall : 0.8160 F1-score : 0.7972 	Accuracy : 0.7625 Precision : 0.7600 Recall : 0.8066 F1-score : 0.7826 	Accuracy : 0.3925 Precision : 0.4286 Recall : 0.4387 F1-score : 0.4336 

logistic_data_C

0.1	0.01	0.001
Accuracy : 0.9762 Precision : 0.9852 Recall : 0.9719	Accuracy : 0.9456 Precision : 0.9707 Recall : 0.9303	Accuracy : 0.8406 Precision : 0.8774 Recall : 0.8290

F1-score : 0.9785 	F1-score : 0.9500 	F1-score : 0.8525 

logistic_data_D

0.1	0.01	0.001
Accuracy : 0.9225 Precision : 0.9330 Recall : 0.9266 F1-score : 0.9298 	Accuracy : 0.9113 Precision : 0.9346 Recall : 0.9029 F1-score : 0.9185 	Accuracy : 0.8319 Precision : 0.8668 Recall : 0.8228 F1-score : 0.8442 

c. Describe your observations in 2(a) and 2(b).

Similar to what I observed in 1(a) and 1(b):

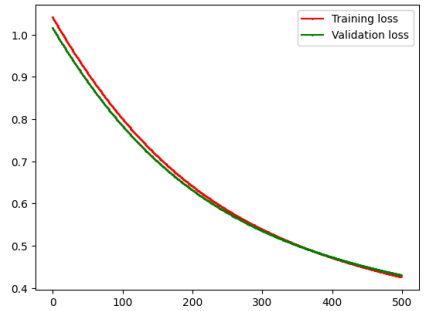
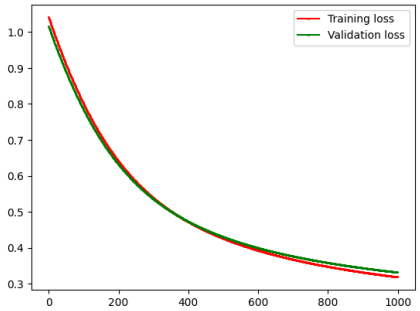
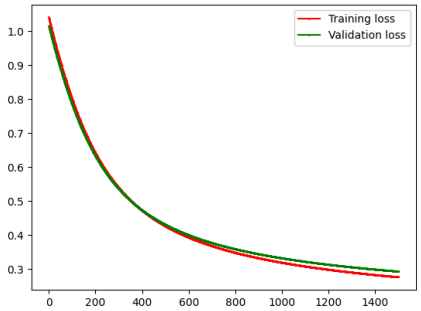
We can see in 2(a) that the loss is in general (for each learning rate) the highest for logistic_data_B. And indeed, the difference is confirmed by the 2(b) stats, since for logistic_data_B, the accuracy is lower.

Plus, we notice that the higher the learning rate, the better the precision. This makes sense since the model learns faster with a higher learning rate; thus, it fits the training data faster, which leads to greater accuracy.

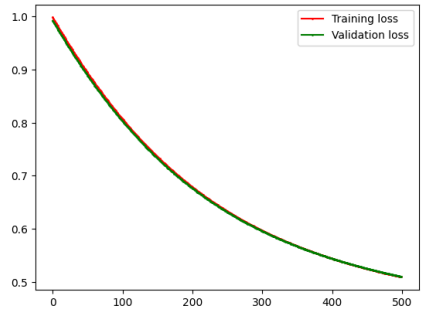
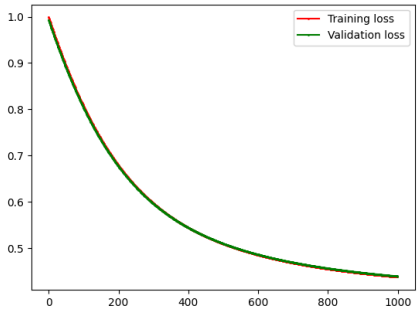
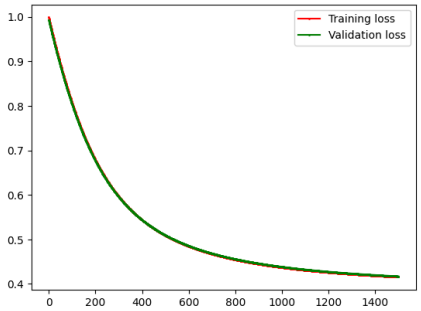
d. Change the number of training iterations to the following values, and plot the loss curve for each setting (using a fixed learning rate of 0.01):

n_iteration∈{500, 1000, 1500}

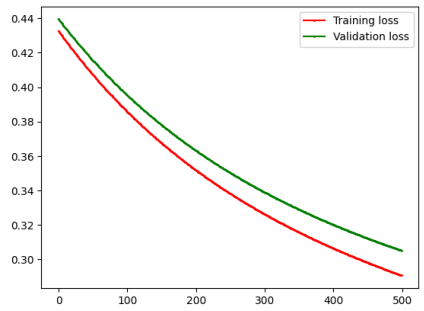
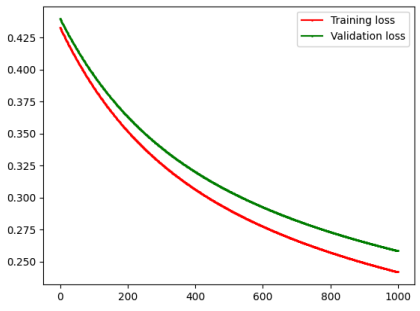
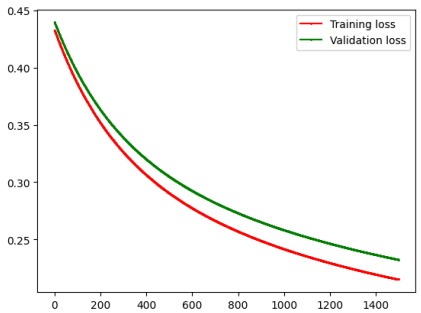
logistic_data_A

500	1000	1500
		

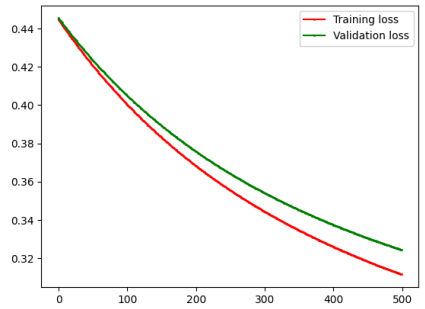
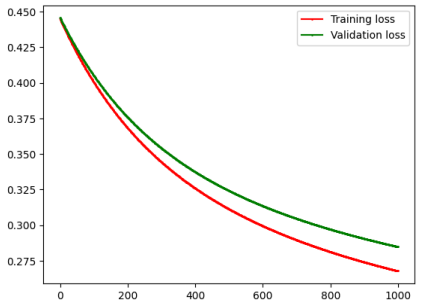
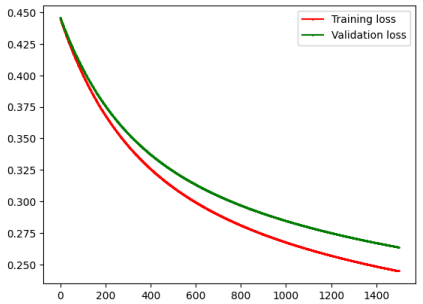
logistic_data_B

500	1000	1500
		

logistic_data_C

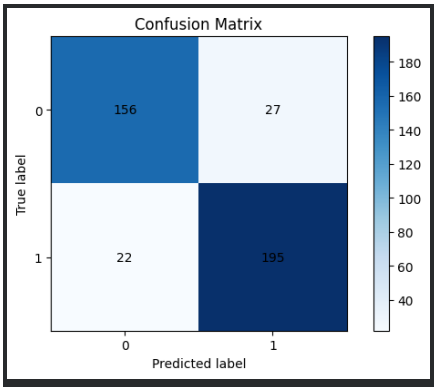
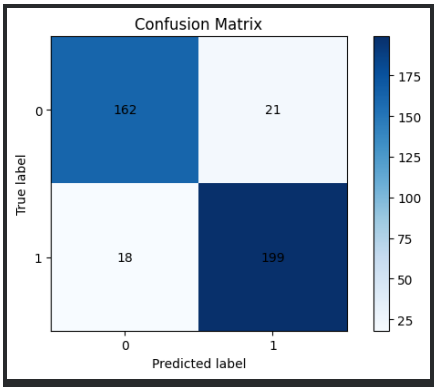
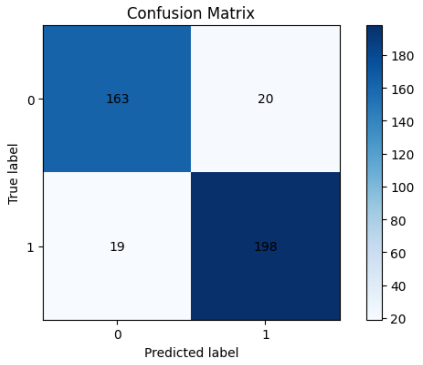
500	1000	1500
		

logistic_data_D

500	1000	1500
		

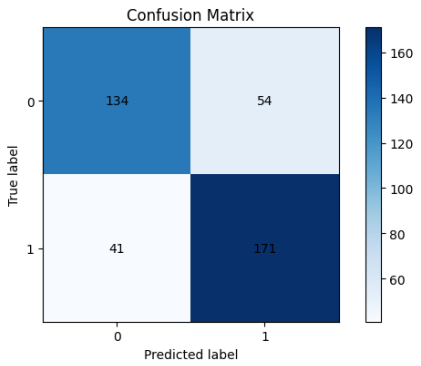
e. Following 2(d), show the evaluations by using the "evaluate_binary_classifier" function.

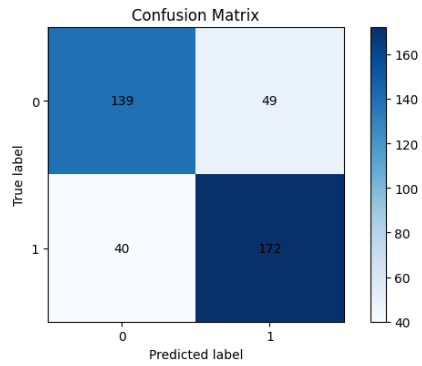
logistic_data_A

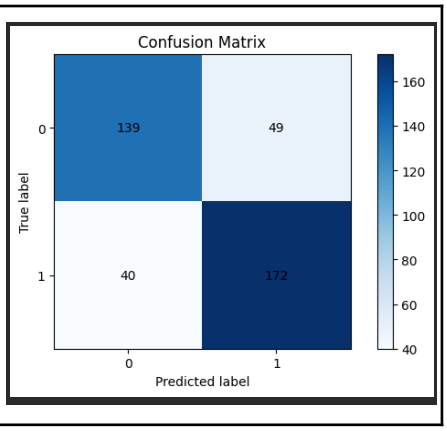
500	1000	1500
Accuracy : 0.8775 Precision : 0.8784 Recall : 0.8986 F1-score : 0.8884 	Accuracy : 0.9025 Precision : 0.9045 Recall : 0.9171 F1-score : 0.9108 	Accuracy : 0.9025 Precision : 0.9083 Recall : 0.9124 F1-score : 0.9103 

logistic_data_B

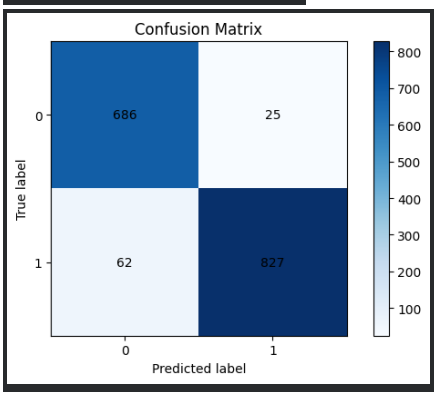
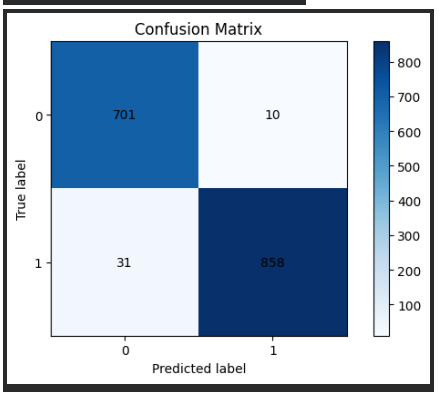
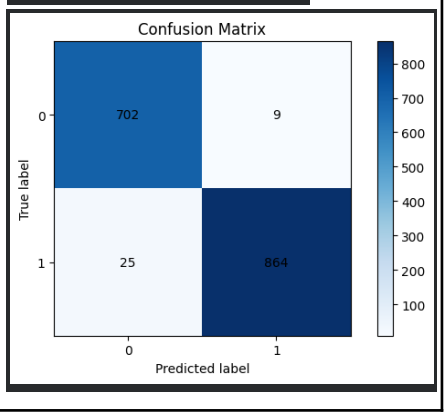
500	1000	1500
Accuracy : 0.7625 Precision : 0.7600 Recall : 0.8066 F1-score : 0.7826	Accuracy : 0.7775 Precision : 0.7783 Recall : 0.8113 F1-score : 0.7945	Accuracy : 0.7775 Precision : 0.7783 Recall : 0.8113 F1-score : 0.7945





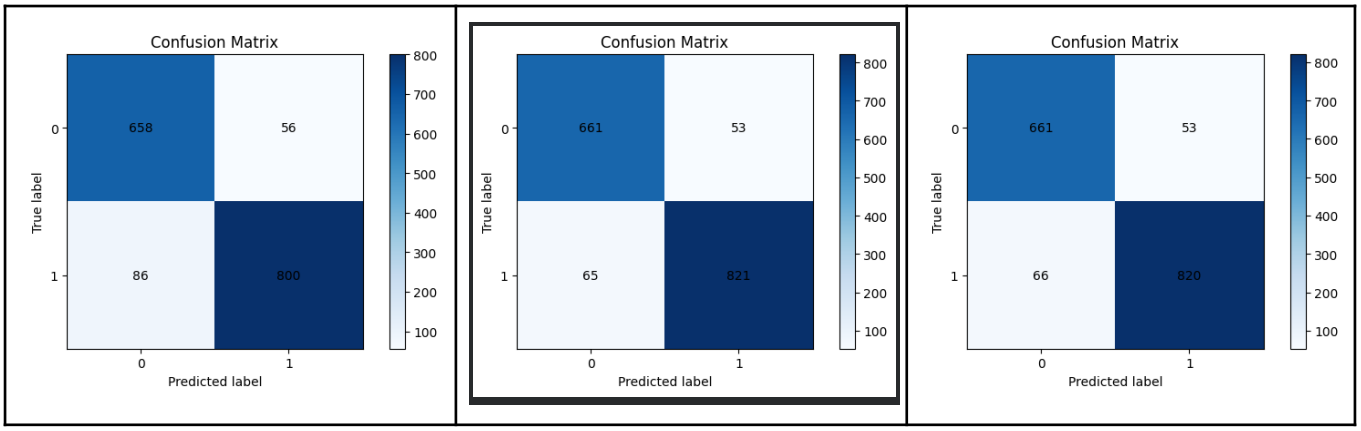


logistic_data_C

500	1000	1500
Accuracy : 0.9456 Precision : 0.9707 Recall : 0.9303 F1-score : 0.9500	Accuracy : 0.9744 Precision : 0.9885 Recall : 0.9651 F1-score : 0.9767	Accuracy : 0.9788 Precision : 0.9897 Recall : 0.9719 F1-score : 0.9807
		

logistic_data_D

500	1000	1500
Accuracy : 0.9113 Precision : 0.9346 Recall : 0.9029 F1-score : 0.9185	Accuracy : 0.9263 Precision : 0.9394 Recall : 0.9266 F1-score : 0.9330	Accuracy : 0.9256 Precision : 0.9393 Recall : 0.9255 F1-score : 0.9323



f. Describe your observations in 2(d) and 2(e).

The greater the number of iterations, the better the result (lower loss and greater accuracy). This makes sense, since the more the model is trained, the better it will fit the data. Also, the training effect is best in the beginning; then, the model slowly converges.

3. Data Preprocessing (20 points):

Please refer to **Real_World_Classification.ipynb**. **Do not change random_seed**, as it will affect the evaluation results. NYCU_Iris.csv is in the data folder.

a. Report the **median** and **standard deviation** for columns with missing values.

```
before imputation:
SepalLengthCm | median: 6.3                | standard deviation: 1.03711450186469
SepalWidthCm  | median: 2.9                | standard deviation: 0.38956399923008
PetalLengthCm | median: 5.085611510791367 | standard deviation: 1.58280738405637
PetalWidthCm  | median: 1.6                | standard deviation: 0.70668935190218
BranchLength  | median: 16.3               | standard deviation: 1.03522779832616

SepalLengthCm | mean: 6.3205508474576275 | standard deviation: 1.037114501864691
SepalWidthCm  | mean: 2.8374717832957113 | standard deviation: 0.389563999230080
PetalLengthCm | mean: 5.085611510791367  | standard deviation: 1.582807384056378
PetalWidthCm  | mean: 1.6757322175732217 | standard deviation: 0.706689351902181
BranchLength  | mean: 16.30294117647059  | standard deviation: 1.035227798326167
```

b. Write the code to **replace the missing values** using "**Nearest Neighbors Imputation**" for columns with missing values. Report the **median** and **standard deviation** of those columns. Compare your result with 3(a) and **explain your observation**.

```
after imputation:
SepalLengthCm | median: 6.3                | standard deviation: 1.02847356589827
SepalWidthCm  | median: 2.88               | standard deviation: 0.37519647010914
PetalLengthCm | median: 5.085611510791367 | standard deviation: 1.45852516657928
PetalWidthCm  | median: 1.6                | standard deviation: 0.69403566234106
```

```
BranchLength  | median: 16.3                | standard deviation: 1.02944016071308
```

```
SepalLengthCm | mean: 6.2982000000000005 | standard deviation: 1.028473565898271
SepalWidthCm  | mean: 2.83756            | standard deviation: 0.375196470109148
PetalLengthCm | mean: 5.141202877697842  | standard deviation: 1.458525166579284
PetalWidthCm  | mean: 1.66948            | standard deviation: 0.694035662341065
BranchLength  | mean: 16.28364           | standard deviation: 1.029440160713081
```

We can see that the median and the standard deviation changed only a little bit (and even didn't change at all for 4 medians out of 5).
This is not very surprising because the nearest neighbors imputation fills the missing value with the mean of the 5 values already existing in the list, which is the value of the 5 closest neighbors.

(I saw in the start "mean" instead of "median"…)
I calculated the percentage of missing values and also the ratio of difference for mean and standard deviation before and after imputation.

```
SepalLengthCm | proportion of missing values: 5.60%
SepalWidthCm  | proportion of missing values: 11.40%
PetalLengthCm | proportion of missing values: 30.00%
PetalWidthCm  | proportion of missing values: 4.40%
BranchLength  | proportion of missing values: 4.80%
```

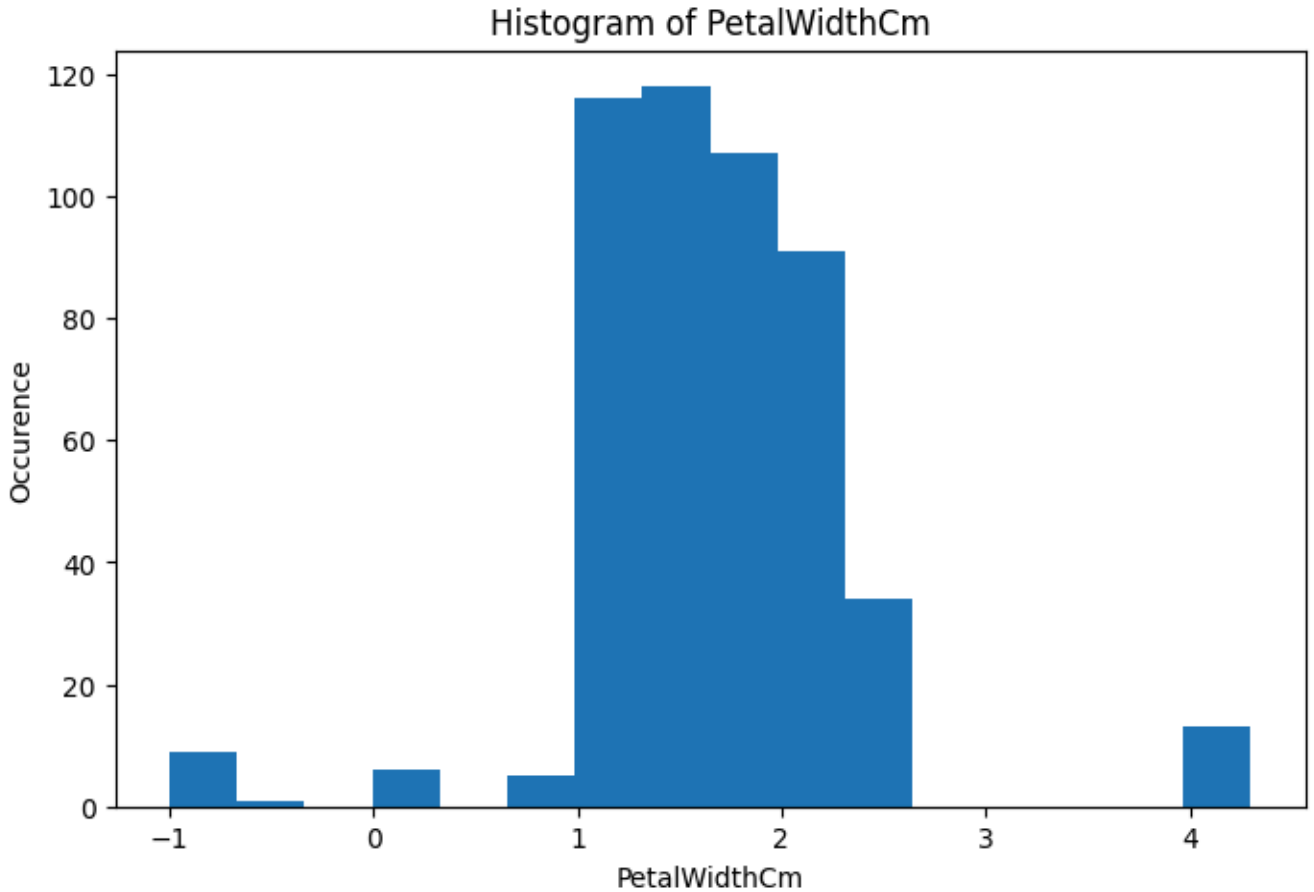
```
imputation difference:
SepalLengthCm | Δmean / mean: 0.354% | Δstd / std: 0.833%
SepalWidthCm  | Δmean / mean: 0.003% | Δstd / std: 3.688%
PetalLengthCm | Δmean / mean: 1.093% | Δstd / std: 7.852%
PetalWidthCm  | Δmean / mean: 0.373% | Δstd / std: 1.791%
BranchLength  | Δmean / mean: 0.118% | Δstd / std: 0.559%
```

We can see that the mean and the standard deviation changed only a little bit. The change was the most significant for PetalLengthCm, which is not surprising since it has 30% missing values.

4. Data Exploration (20 points):

Please continue using **Real_World_Classification.ipynb**. Based on the preprocessed data from Task 3, proceed with the data exploration. Do not change the random_seed to ensure consistency in evaluation results.

a. **Plot the histogram** of the column "PetalWidthCm" in "NYCU_Iris.csv". Please **describe your observations** for the histogram.



The PetalWidthCm histogram is centered around the value of 1.5 cm, which confirms the value of the median (and also the mean), and we can also see the small standard deviation through the peak of values around the mean.
However, we can see that there are approximately 10 negative values, which shouldn't exist. This should be an error while entering the values (the nearest neighbors imputation method doesn't produce values outside the max and min).

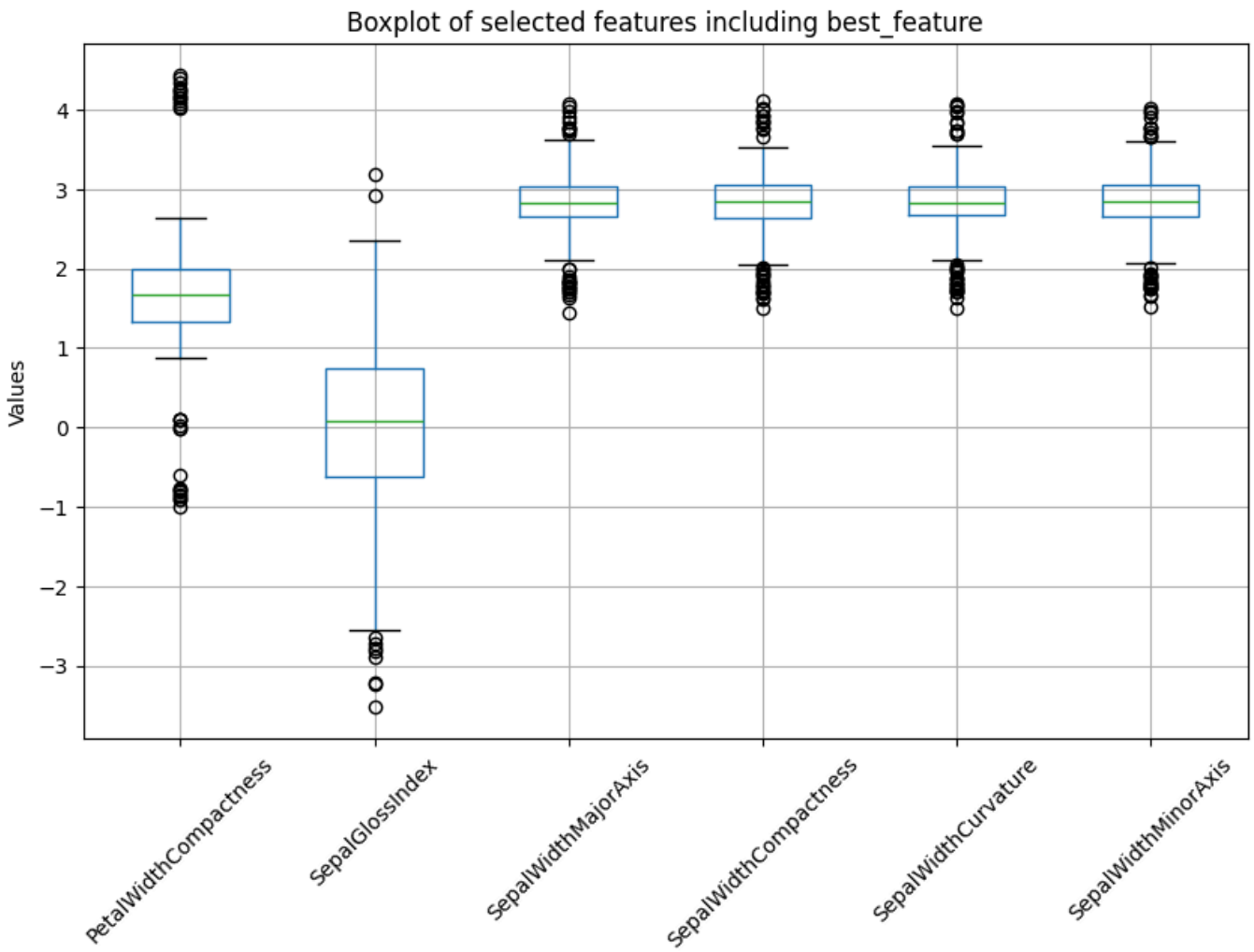
b. **List the feature** (excluding "PetalWidthCm", "Species", and "Id") that has the **largest positive correlation** (Pearson correlation coefficient) **with the "PetalWidthCm"** column.

```
Feature with highest positive correlation:
PetalWidthCompactness | correlation: 0.9925409158885194
```

c. **Report the top 5 features** with the **strongest negative** correlations with the 'PetalWidthCm' column.

```
SepalGlossIndex       | correlation: -0.10589317210333306
SepalWidthMajorAxis   | correlation: -0.08177284310553865
SepalWidthCompactness | correlation: -0.07310628921187612
SepalWidthCurvature   | correlation: -0.0679294627746147
SepalWidthMinorAxis   | correlation: -0.06185278055464031
```

d. **Plot the boxplot** for all features found in 4(b) and 4(c).

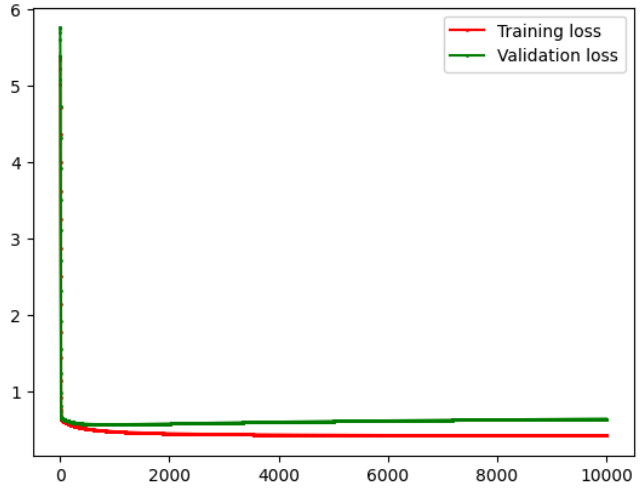
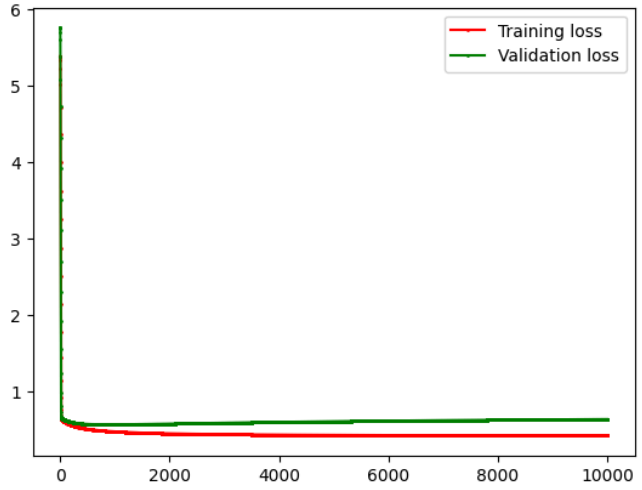
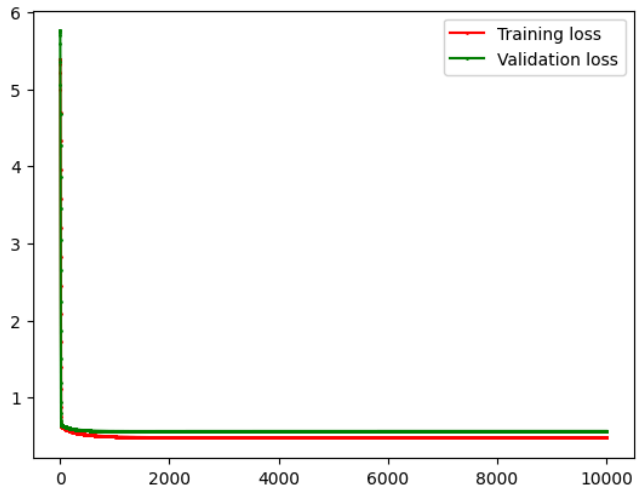
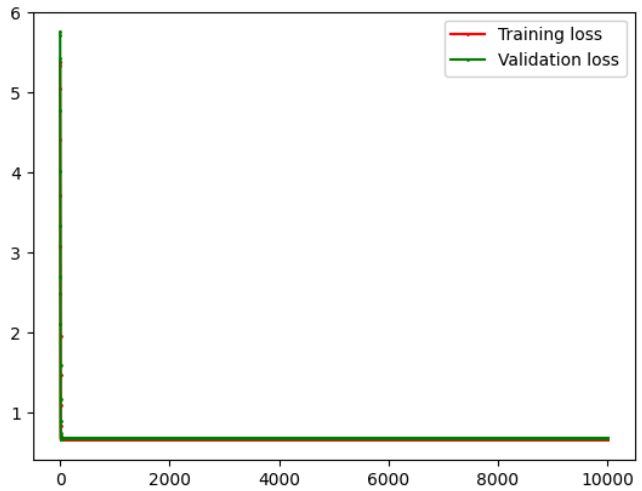


5. Regularization (20 points):

Please continue using **Real_World_Classification.ipynb**. Based on the preprocessed data from Task 3, proceed with the data exploration. Do not change the random_seed to ensure consistency in evaluation results.

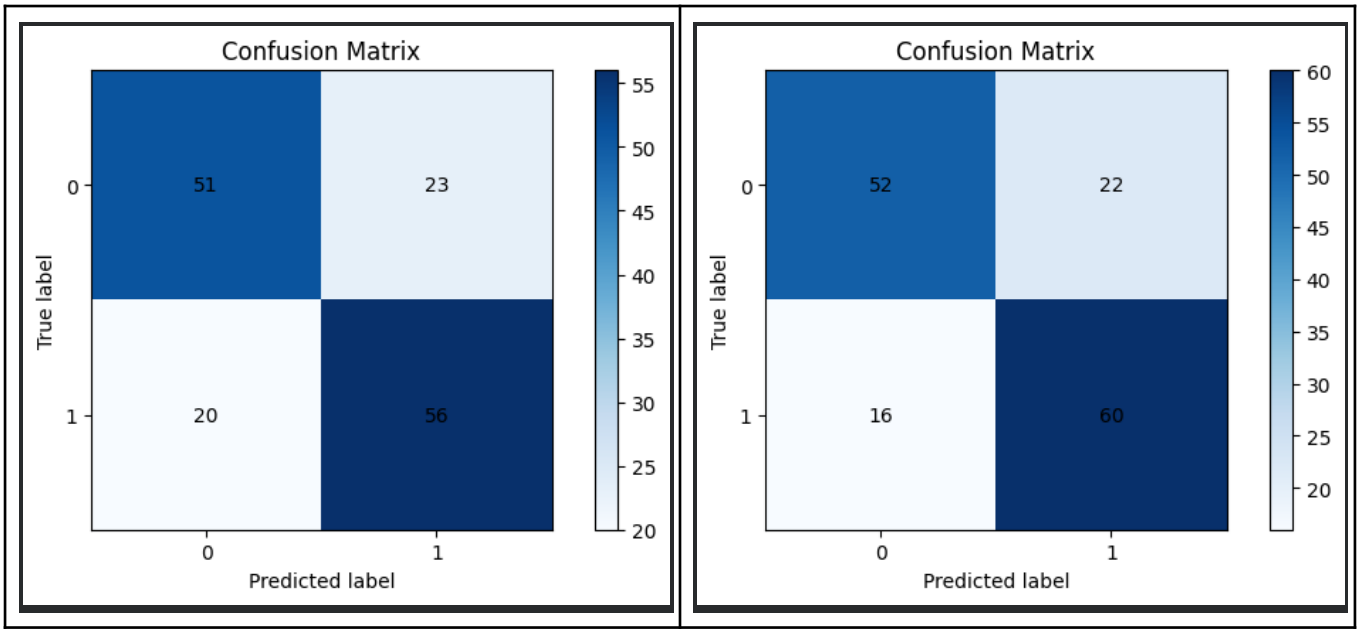
a. **L2 Regularization.** Please run Real_World_Classification.ipynb to compare the performance between no regularization (current code) and L2 regularization. For L2 regularization, set the regularization parameter (reg_lambda in the code) to 0.01, 1, and 100, respectively. Then, **plot the loss curves** for all settings:

(1) No regularization
(2) L2 (reg_lambda=0.01)
(3) L2 (reg_lambda=1)
(4) L2 (reg_lambda=100)

No regularization	L2 (reg_lambda=0.01)
	
L2 (reg_lambda=0.01)	L2 (reg_lambda=100)
	

b. Following 5(a), show the testing results by using the "evaluate_binary_classifier" function.

No regularization	L2 (reg_lambda=1)
Accuracy : 0.7133 Precision : 0.7089 Recall : 0.7368 F1-score : 0.7226	Accuracy : 0.7467 Precision : 0.7317 Recall : 0.7895 F1-score : 0.7595



c. Describe your observations in 5(a) and 5(b).

We can see in 5(a) that the greater the lambda value we use with the L2 regularization, the closer the value between validation loss and training loss.

And it is normal, since the purpose of the regularization is to avoid overfitting by giving a disadvantage to big model weights:

```
Sum of absolute values of weights: 59.40219576564125
```

```
Sum of absolute values of weights: 24.750200852653265
```

And this is also confirmed in 5(b), the moderated regularization has better accuracy.